

COMP5318 Assignment 2: Image Classification

Group number: 153

SID1: 540430668 SID2: 540776016 SID3: 520197321 SID4: ...

This template notebook includes code to load the dataset and a skeleton for the main sections that should be included in the notebook. Please stick to this structure for your submitted notebook.

Please focus on making your code clear, with appropriate variable names and whitespace. Include comments and markdown text to aid the readability of your code where relevant. See the specification and marking criteria in the associated specification to guide you when completing your implementation.

Setup and dependencies

Please use this section to list and set up all your required libraries/dependencies and your plotting environment.

```
In [1]: # Install core dependencies for the notebook
!pip install numpy pandas matplotlib seaborn scikit-learn tensorflow
```

```
Requirement already satisfied: numpy in /opt/anaconda3/envs/tencent/
lib/python3.10/site-packages (2.2.6)
Requirement already satisfied: pandas in /opt/anaconda3/envs/tencen
t/lib/python3.10/site-packages (2.3.3)
Collecting matplotlib
  Downloading matplotlib-3.10.9-cp310-cp310-macosx_11_0_arm64.whl.me
tadata (52 kB)
Collecting seaborn
  Downloading seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Requirement already satisfied: scikit-learn in /opt/anaconda3/envs/t
encent/lib/python3.10/site-packages (1.7.2)
Collecting tensorflow
  Downloading tensorflow-2.21.0-cp310-cp310-macosx_12_0_arm64.whl.me
tadata (4.4 kB)
Collecting keras-tuner
  Using cached keras_tuner-1.4.8-py3-none-any.whl.metadata (5.6 kB)
Requirement already satisfied: python-dateutil>=2.8.2 in /opt/anacon
da3/envs/tencent/lib/python3.10/site-packages (from pandas) (2.9.0.p
ost0)
Requirement already satisfied: pytz>=2020.1 in /opt/anaconda3/envs/t
encent/lib/python3.10/site-packages (from pandas) (2026.1.post1)
Requirement already satisfied: tzdata>=2022.7 in /opt/anaconda3/env
s/tencent/lib/python3.10/site-packages (from pandas) (2026.2)
```

```
Collecting contourpy>=1.0.1 (from matplotlib)
  Downloading contourpy-1.3.2-cp310-cp310-macosx_11_0_arm64.whl.metadata (5.5 kB)
Collecting cycler>=0.10 (from matplotlib)
  Downloading cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.22.0 (from matplotlib)
  Downloading fonttools-4.62.1-cp310-cp310-macosx_10_9_universal2.whl.metadata (117 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib)
  Downloading kiwisolver-1.5.0-cp310-cp310-macosx_11_0_arm64.whl.metadata (5.1 kB)
Requirement already satisfied: packaging>=20.0 in /opt/anaconda3/envs/tencent/lib/python3.10/site-packages (from matplotlib) (26.0)
Collecting pillow>=8 (from matplotlib)
  Downloading pillow-12.2.0-cp310-cp310-macosx_11_0_arm64.whl.metadata (8.8 kB)
Collecting pyparsing>=3 (from matplotlib)
  Downloading pyparsing-3.3.2-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: scipy>=1.8.0 in /opt/anaconda3/envs/tencent/lib/python3.10/site-packages (from matplotlib) (1.15.3)
Requirement already satisfied: joblib>=1.2.0 in /opt/anaconda3/envs/tencent/lib/python3.10/site-packages (from matplotlib) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /opt/anaconda3/envs/tencent/lib/python3.10/site-packages (from matplotlib) (3.6.0)
Collecting absl-py>=1.0.0 (from tensorflow)
  Using cached absl_py-2.4.0-py3-none-any.whl.metadata (3.3 kB)
Collecting astunparse>=1.6.0 (from tensorflow)
  Using cached astunparse-1.6.3-py2.py3-none-any.whl.metadata (4.4 kB)
Collecting flatbuffers>=25.9.23 (from tensorflow)
  Using cached flatbuffers-25.12.19-py2.py3-none-any.whl.metadata (1.0 kB)
Collecting gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 (from tensorflow)
  Using cached gast-0.7.0-py3-none-any.whl.metadata (1.5 kB)
Collecting google_pasta>=0.1.1 (from tensorflow)
  Using cached google_pasta-0.2.0-py3-none-any.whl.metadata (814 bytes)
Collecting libclang>=13.0.0 (from tensorflow)
  Using cached libclang-18.1.1-1-py2.py3-none-macosx_11_0_arm64.whl.metadata (5.2 kB)
Collecting opt_einsum>=2.3.2 (from tensorflow)
  Using cached opt_einsum-3.4.0-py3-none-any.whl.metadata (6.3 kB)
Collecting protobuf<8.0.0,>=6.31.1 (from tensorflow)
  Downloading protobuf-7.34.1-cp310-abi3-macosx_10_9_universal2.whl.metadata (595 bytes)
Collecting requests<3,>=2.21.0 (from tensorflow)
  Downloading requests-2.33.1-py3-none-any.whl.metadata (4.8 kB)
Requirement already satisfied: setuptools in /opt/anaconda3/envs/tencent/lib/python3.10/site-packages (from tensorflow) (81.0.0)
Requirement already satisfied: six>=1.12.0 in /opt/anaconda3/envs/tencent/lib/python3.10/site-packages (from tensorflow) (1.17.0)
Collecting termcolor>=1.1.0 (from tensorflow)
  Using cached termcolor-3.3.0-py3-none-any.whl.metadata (6.5 kB)
Requirement already satisfied: typing_extensions>=3.6.6 in /opt/anaconda3/envs/tencent/lib/python3.10/site-packages (from tensorflow) (4.12.0)
```

4.15.0)

Collecting wrapt<=1.11.0 (from tensorflow)

Downloading wrapt-2.1.2-cp310-cp310-macosx_11_0_arm64.whl.metadata (7.4 kB)

Collecting grpcio<2.0,>=1.24.3 (from tensorflow)

Downloading grpcio-1.80.0-cp310-cp310-macosx_11_0_universal2.whl.metadata (3.8 kB)

Collecting keras<=3.12.0 (from tensorflow)

Downloading keras-3.12.2-py3-none-any.whl.metadata (5.9 kB)

Collecting h5py<3.15.0,>=3.11.0 (from tensorflow)

Downloading h5py-3.14.0-cp310-cp310-macosx_11_0_arm64.whl.metadata (2.7 kB)

Collecting ml_dtypes<1.0.0,>=0.5.1 (from tensorflow)

Downloading ml_dtypes-0.5.4-cp310-cp310-macosx_10_9_universal2.whl.metadata (8.9 kB)

Collecting charset_normalizer<4,>=2 (from requests<3,>=2.21.0->tensorflow)

Downloading charset_normalizer-3.4.7-cp310-cp310-macosx_10_9_universal2.whl.metadata (40 kB)

Collecting idna<4,>=2.5 (from requests<3,>=2.21.0->tensorflow)

Downloading idna-3.14-py3-none-any.whl.metadata (8.0 kB)

Collecting urllib3<3,>=1.26 (from requests<3,>=2.21.0->tensorflow)

Downloading urllib3-2.7.0-py3-none-any.whl.metadata (6.9 kB)

Collecting certifi>=2023.5.7 (from requests<3,>=2.21.0->tensorflow)

Downloading certifi-2026.4.22-py3-none-any.whl.metadata (2.5 kB)

Collecting kt-legacy (from keras-tuner)

Using cached kt_legacy-1.0.5-py3-none-any.whl.metadata (221 bytes)

Requirement already satisfied: wheel<1.0,>=0.23.0 in /opt/anaconda3/envs/tencent/lib/python3.10/site-packages (from astunparse>=1.6.0->tensorflow) (0.46.3)

Collecting rich (from keras>=3.12.0->tensorflow)

Downloading rich-15.0.0-py3-none-any.whl.metadata (18 kB)

Collecting namex (from keras>=3.12.0->tensorflow)

Using cached namex-0.1.0-py3-none-any.whl.metadata (322 bytes)

Collecting optree (from keras>=3.12.0->tensorflow)

Downloading optree-0.19.1-cp310-cp310-macosx_11_0_arm64.whl.metadata (32 kB)

Collecting markdown-it-py>=2.2.0 (from rich->keras>=3.12.0->tensorflow)

Downloading markdown_it_py-4.2.0-py3-none-any.whl.metadata (7.4 kB)

Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /opt/anaconda3/envs/tencent/lib/python3.10/site-packages (from rich->keras>=3.12.0->tensorflow) (2.20.0)

Collecting mdurl~0.1 (from markdown-it-py>=2.2.0->rich->keras>=3.12.0->tensorflow)

Downloading mdurl-0.1.2-py3-none-any.whl.metadata (1.6 kB)

Downloading matplotlib-3.10.9-cp310-cp310-macosx_11_0_arm64.whl (8.2 MB)

8.2/8.2 MB 22.2 MB/s 0:00:00 eta 0:00:01

Downloading seaborn-0.13.2-py3-none-any.whl (294 kB)

Downloading tensorflow-2.21.0-cp310-cp310-macosx_12_0_arm64.whl (23.2 MB)

223.2/223.2 MB 8.3 MB/s 0:00:26m0:00:0100:01

```

Downloading grpcio-1.80.0-cp310-cp310-macosx_11_0_universal2.whl (1
2.0 MB)
_____ 12.0/12.0 MB 38.2 MB/s
0:00:00eta 0:00:01
Downloading h5py-3.14.0-cp310-cp310-macosx_11_0_arm64.whl (2.8 MB)
_____ 2.8/2.8 MB 9.2 MB/s 0:0
0:00m eta 0:00:01
Downloading ml_dtypes-0.5.4-cp310-cp310-macosx_10_9_universal2.whl (
679 kB)
_____ 679.7/679.7 kB 21.3 MB/s
0:00:00
Downloading protobuf-7.34.1-cp310-abi3-macosx_10_9_universal2.whl (4
29 kB)
Downloading requests-2.33.1-py3-none-any.whl (64 kB)
Downloading charset_normalizer-3.4.7-cp310-cp310-macosx_10_9_univers
al2.whl (315 kB)
Downloading idna-3.14-py3-none-any.whl (72 kB)
Downloading urllib3-2.7.0-py3-none-any.whl (131 kB)
Using cached keras_tuner-1.4.8-py3-none-any.whl (129 kB)
Using cached absl_py-2.4.0-py3-none-any.whl (135 kB)
Using cached astunparse-1.6.3-py2.py3-none-any.whl (12 kB)
Downloading certifi-2026.4.22-py3-none-any.whl (135 kB)
Downloading contourpy-1.3.2-cp310-cp310-macosx_11_0_arm64.whl (253 k
B)
Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Using cached flatbuffers-25.12.19-py2.py3-none-any.whl (26 kB)
Downloading fonttools-4.62.1-cp310-cp310-macosx_10_9_universal2.whl
(2.9 MB)
_____ 2.9/2.9 MB 34.3 MB/s 0:
00:00
Using cached gast-0.7.0-py3-none-any.whl (22 kB)
Using cached google_pasta-0.2.0-py3-none-any.whl (57 kB)
Downloading keras-3.12.2-py3-none-any.whl (1.5 MB)
_____ 1.5/1.5 MB 31.5 MB/s 0:
00:00
Downloading kiwisolver-1.5.0-cp310-cp310-macosx_11_0_arm64.whl (63 k
B)
Using cached libclang-18.1.1-1-py2.py3-none-macosx_11_0_arm64.whl (2
5.8 MB)
Using cached opt_einsum-3.4.0-py3-none-any.whl (71 kB)
Downloading pillow-12.2.0-cp310-cp310-macosx_11_0_arm64.whl (4.7 MB)
_____ 4.7/4.7 MB 38.0 MB/s 0:
00:00
Downloading pyparsing-3.3.2-py3-none-any.whl (122 kB)
Using cached termcolor-3.3.0-py3-none-any.whl (7.7 kB)
Downloading wrapt-2.1.2-cp310-cp310-macosx_11_0_arm64.whl (61 kB)
Using cached kt_legacy-1.0.5-py3-none-any.whl (9.6 kB)
Using cached namex-0.1.0-py3-none-any.whl (5.9 kB)
Downloading optree-0.19.1-cp310-cp310-macosx_11_0_arm64.whl (370 kB)
Downloading rich-15.0.0-py3-none-any.whl (310 kB)
Downloading markdown_it_py-4.2.0-py3-none-any.whl (91 kB)
Downloading mdurl-0.1.2-py3-none-any.whl (10.0 kB)
Installing collected packages: namex, libclang, kt-legacy, flatbuffe
rs, wrapt, urllib3, termcolor, pyparsing, protobuf, pillow, optree,
opt_einsum, ml_dtypes, mdurl, kiwisolver, idna, h5py, grpcio, google
_pasta, gast, fonttools, cycler, contourpy, charset_normalizer, cert

```

ifi, absl-py, requests, matplotlib, markdown-it-py, astunparse, seaborn, rich, keras, tensorflow, keras-tuner

35/35 [keras-tuner] [tensorflow]

Successfully installed absl-py-2.4.0 astunparse-1.6.3 certifi-2026.4.22 charset_normalizer-3.4.7 contourpy-1.3.2 cycycler-0.12.1 flatbuffers-25.12.19 fonttools-4.62.1 gast-0.7.0 google_pasta-0.2.0 grpcio-1.80.0 h5py-3.14.0 idna-3.14 keras-3.12.2 keras-tuner-1.4.8 kiwisolver-1.5.0 kt-legacy-1.0.5 libclang-18.1.1 markdown-it-py-4.2.0 matplotlib-3.10.9 mdurl-0.1.2 ml_dtypes-0.5.4 namex-0.1.0 opt_einsum-3.4.0 optree-0.19.1 pillow-12.2.0 protobuf-7.34.1 pyparsing-3.3.2 requests-2.33.1 rich-15.0.0 seaborn-0.13.2 tensorflow-2.21.0 termcolor-3.3.0 urllib3-2.7.0 wrapt-2.1.2

```
In [2]: import sys
import subprocess

import os
os.environ["TF_USE_LEGACY_KERAS"] = "0"
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"

import time
import pandas as pd

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

from sklearn.metrics import accuracy_score, classification_report,

from pathlib import Path

import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
from sklearn.model_selection import train_test_split, ParameterGrid
from sklearn.preprocessing import StandardScaler

import keras_tuner as kt

USE_CUML = False
try:
    from cuml.linear_model import LogisticRegression
    USE_CUML = True
    print("GPU enabled for LogisticRegression using CuML")
except ImportError:
    from sklearn.linear_model import LogisticRegression
    print("GPU not available, using CPU for LogisticRegression")

HAS_CUPY = False
try:
    import cupy as cp
    HAS_CUPY = cp.cuda.is_available()
except ImportError:
    try:
        subprocess.check_call(
```

```

        [sys.executable, "-m", "pip", "install", "cupy-cuda12x"
        stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL,
        )
    import cupy as cp
    HAS_CUPY = cp.cuda.is_available()
except Exception:
    pass
print(f"CuPy GPU acceleration: {'enabled' if HAS_CUPY else 'not available'}")

plt.style.use('seaborn-v0_8-whitegrid')
plt.rcParams['figure.dpi'] = 120
RANDOM_STATE = 5318

processed_dir = Path('processed')
output_dir = Path('output')
processed_dir.mkdir(exist_ok=True)
output_dir.mkdir(exist_ok=True)

```

GPU not available, using CPU for LogisticRegression
 CuPy GPU acceleration: not available

1. Data loading, exploration, and preprocessing

Code to load the dataset is provided in the following cell. Please proceed with your data exploration and preprocessing in the remainder of this section.

```

In [3]: # Load the dataset training and test sets as numpy arrays
        # assuming Assignment2Data folder is present in the same directory
        # as the notebook
X_train = np.load('Assignment2Data/X_train.npy')
y_train = np.load('Assignment2Data/y_train.npy')
X_test = np.load('Assignment2Data/X_test.npy')
y_test = np.load('Assignment2Data/y_test.npy')

```

Dataset overview

Start by checking dataset size, datatype, and label coverage before plotting.

```

In [4]: print(f'X_train shape: {X_train.shape}, dtype: {X_train.dtype}')
        print(f'y_train shape: {y_train.shape}, dtype: {y_train.dtype}')
        print(f'X_test shape: {X_test.shape}, dtype: {X_test.dtype}')
        print(f'y_test shape: {y_test.shape}, dtype: {y_test.dtype}')
        print(f'Pixel range in training data: [{X_train.min()}, {X_train.max()}]')
        print(f'Unique class labels: {np.unique(y_train)}')

```

```

X_train shape: (32000, 28, 28, 3), dtype: uint8
y_train shape: (32000,), dtype: uint8
X_test shape: (8000, 28, 28, 3), dtype: uint8
y_test shape: (8000,), dtype: uint8
Pixel range in training data: [0, 255]
Unique class labels: [0 1 2 3 4 5 6 7 8]

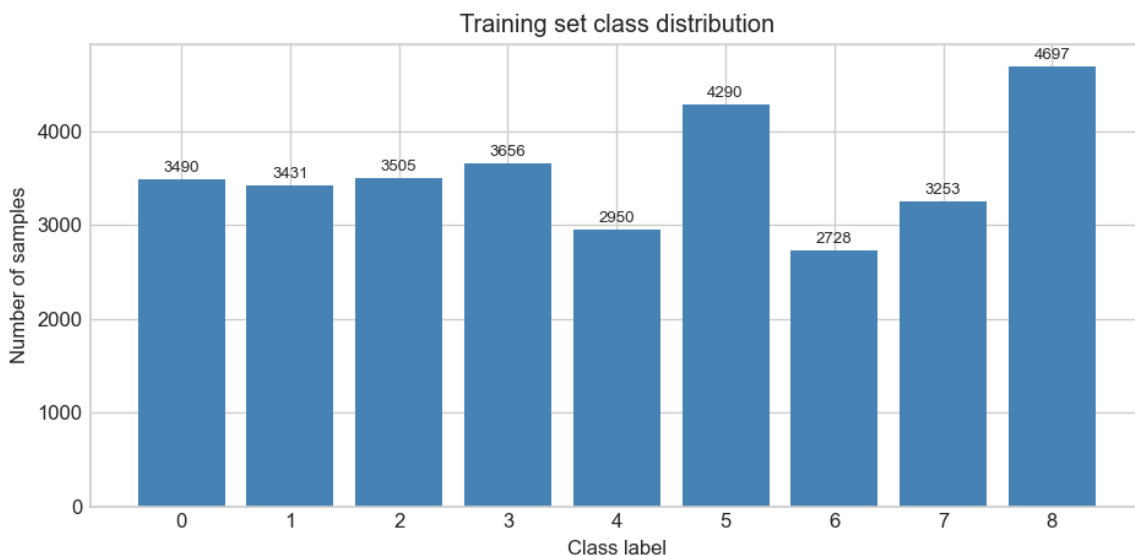
```

Class distribution

This bar chart checks whether the classes are balanced in the training split.

```
In [5]: class_ids, class_counts = np.unique(y_train, return_counts=True)

fig, ax = plt.subplots(figsize=(8, 4))
ax.bar(class_ids.astype(str), class_counts, color='steelblue')
ax.set_title('Training set class distribution')
ax.set_xlabel('Class label')
ax.set_ylabel('Number of samples')
for idx, count in enumerate(class_counts):
    ax.text(idx, count + 40, str(count), ha='center', va='bottom',
fig.tight_layout()
fig.savefig(output_dir / 'class_distribution.png', bbox_inches='tight')
plt.show()
```



Sample images from each class

Display a few examples per class to inspect intra-class variation and class similarity.

```
In [6]: samples_per_class = 4
fig, axes = plt.subplots(len(class_ids), samples_per_class, figsize

for row, class_id in enumerate(class_ids):
    sample_indices = np.where(y_train == class_id)[0][:samples_per_
    for col, sample_idx in enumerate(sample_indices):
        axes[row, col].imshow(X_train[sample_idx])
        axes[row, col].axis('off')
        if col == 0:
            axes[row, col].set_title(f'Class {class_id}', loc='left

fig.suptitle('Training examples by class', fontsize=14)
fig.tight_layout()
fig.savefig(output_dir / 'sample_images_by_class.png', bbox_inches=
plt.show()
```

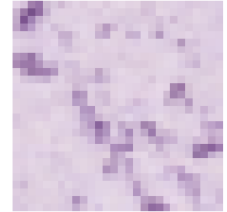
Class 0



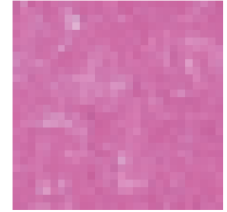
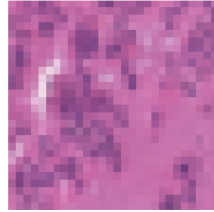
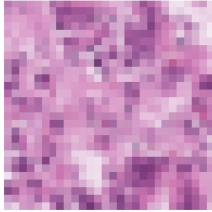
Training examples by class



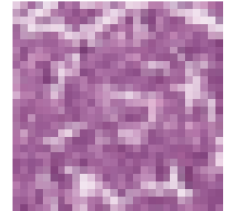
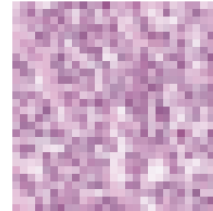
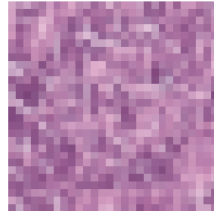
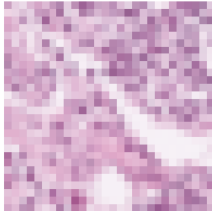
Class 1



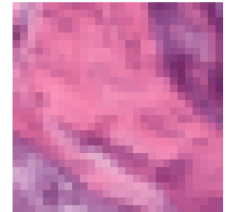
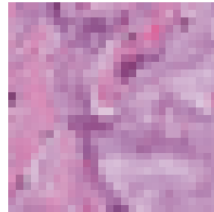
Class 2



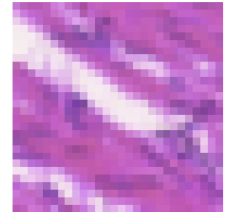
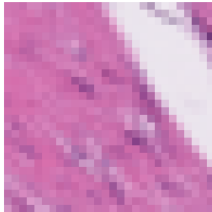
Class 3



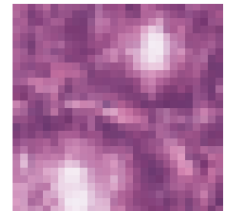
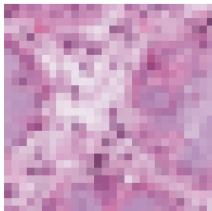
Class 4



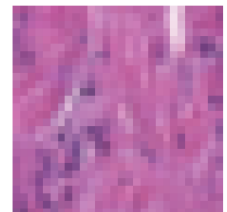
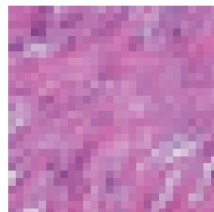
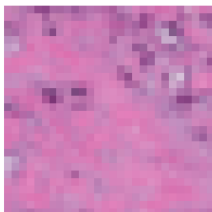
Class 5



Class 6



Class 7



Class 8





Pixel intensity distribution

Plotting the raw pixel values helps justify scaling images to the [0, 1] range.

```
In [7]: fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(X_train.reshape(-1), bins=32, color='darkorange', edgecolor='black')
ax.set_title('Raw pixel intensity distribution (training set)')
ax.set_xlabel('Pixel value')
ax.set_ylabel('Frequency')
fig.tight_layout()
fig.savefig(output_dir / 'pixel_distribution.png', bbox_inches='tight')
plt.show()
```



Preprocessing

The minimal preprocessing used here is image normalisation to float32 values in [0, 1], followed by a stratified train/validation split and a flattened copy for non-CNN models.

```
In [8]: X_train_norm = X_train.astype(np.float32) / 255.0
X_test_norm = X_test.astype(np.float32) / 255.0

X_train_full_flat = X_train_norm.reshape(len(X_train_norm), -1)
X_test_full_flat = X_test_norm.reshape(len(X_test_norm), -1)

X_train_img, X_val_img, y_train_split, y_val = train_test_split(
    X_train_norm,
    y_train,
    test_size=0.2,
    random_state=RANDOM_STATE,
    stratify=y_train)
```

```

)

X_train_flat = X_train_img.reshape(X_train_img.shape[0], -1)
X_val_flat = X_val_img.reshape(X_val_img.shape[0], -1)
X_test_flat = X_test_norm.reshape(X_test_norm.shape[0], -1)

search_scaler = StandardScaler()
X_train_flat_scaled = search_scaler.fit_transform(X_train_flat)
X_val_flat_scaled = search_scaler.transform(X_val_flat)
X_test_flat_scaled = search_scaler.transform(X_test_flat)

full_scaler = StandardScaler()
X_train_full_flat_scaled = full_scaler.fit_transform(X_train_full_f
X_test_full_flat_scaled = full_scaler.transform(X_test_full_flat)

np.save(processed_dir / 'X_train_img.npy', X_train_img)
np.save(processed_dir / 'X_val_img.npy', X_val_img)
np.save(processed_dir / 'X_test_img.npy', X_test_norm)
np.save(processed_dir / 'X_train_flat.npy', X_train_flat)
np.save(processed_dir / 'X_val_flat.npy', X_val_flat)
np.save(processed_dir / 'X_test_flat.npy', X_test_flat)
np.save(processed_dir / 'y_train.npy', y_train_split)
np.save(processed_dir / 'y_val.npy', y_val)
np.save(processed_dir / 'y_test.npy', y_test)

print('Saved processed arrays to:', processed_dir.resolve())
print(f'X_train_img shape: {X_train_img.shape}')
print(f'X_val_img shape: {X_val_img.shape}')
print(f'X_test_img shape: {X_test_norm.shape}')
print(f'X_train_flat shape: {X_train_flat.shape}')

```

Saved processed arrays to: /Users/tangzongquan/Desktop/COMP5318-Assignment-2/processed

X_train_img shape: (25600, 28, 28, 3)

X_val_img shape: (6400, 28, 28, 3)

X_test_img shape: (8000, 28, 28, 3)

X_train_flat shape: (25600, 2352)

Examples of preprocessed data

The figure below compares raw and normalised images. The normalised images look visually similar, but their datatype and numeric range are now more suitable for model training.

```

In [9]: comparison_indices = [0, 1, 2, 3]
fig, axes = plt.subplots(2, len(comparison_indices), figsize=(10, 5)

for col, sample_idx in enumerate(comparison_indices):
    axes[0, col].imshow(X_train[sample_idx])
    axes[0, col].set_title(f'Raw\nlabel={y_train[sample_idx]}')
    axes[0, col].axis('off')

    axes[1, col].imshow(X_train_norm[sample_idx])
    axes[1, col].set_title(f'Normalised\nlabel={y_train[sample_idx]}')

```

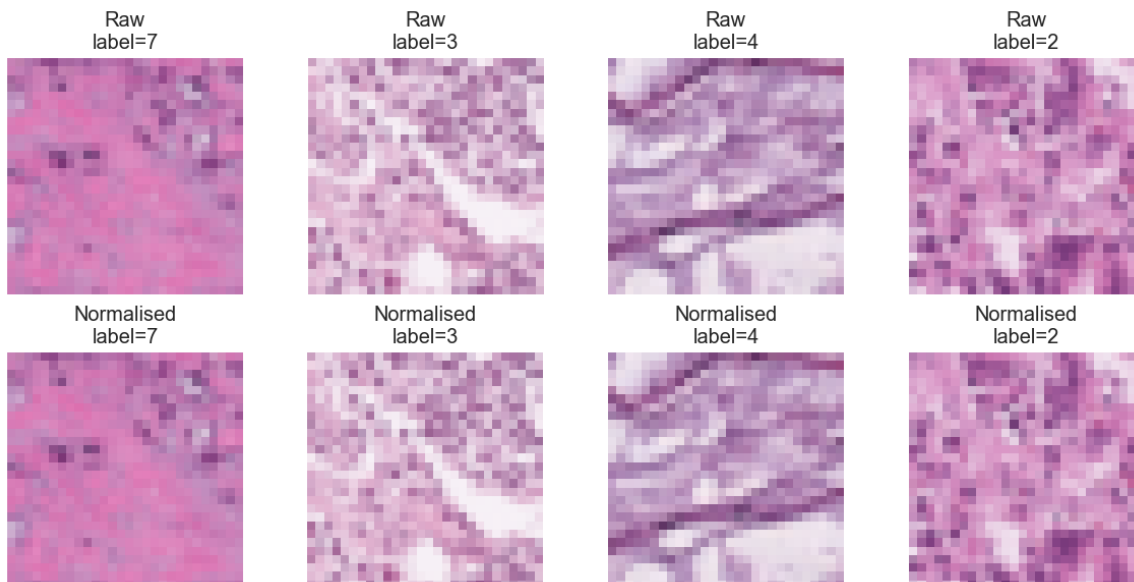
```

axes[1, col].axis('off')

fig.tight_layout()
fig.savefig(output_dir / 'preprocessing_comparison.png', bbox_inches=
plt.show()

print(f'Normalised training range: [{X_train_norm.min():.3f}, {X_train_norm.max():.3f}]')
print(f'Flattened feature count per image: {X_train_flat.shape[1]}')

```



Normalised training range: [0.000, 1.000]
 Flattened feature count per image: 2352

2. Algorithm design and setup

Logistic Regression (Algorithm of choice from first six weeks of course)

The cell below defines the logistic regression model builder and an evaluation helper. The model takes a single key hyperparameter — the regularization strength C (default 1.0) — along with `max_iter=5000` to ensure convergence on the high-dimensional input. By default it uses the `lbfgs` solver on CPU; when GPU acceleration is available, the solver is automatically swapped to CuML's `qn` with L2 penalty. The two paths are functionally equivalent — one is simply optimized for CPU and the other for GPU. Compared to the MLP and CNN used later, logistic regression is a much simpler linear model with far fewer parameters, making it a natural baseline for this assignment. A quick sanity-check fit on 100 samples is also run to confirm the class count, feature dimensionality, and total parameter count.

```

In [10]: # logistic Regression model and evaluation functions

# build a logistic regression model for flattened image vectors.
def build_logistic_regression(C=1.0, solver='lbfgs', class_weight=None,
                              kwargs = {

```

```

        'C': C,
        'max_iter': 5000,
    }

    # GPU-optimized parameters for CuML's qn solver
    if USE_CUML:
        kwargs['solver'] = 'qn'
        kwargs['penalty'] = 'l2'
        kwargs['tol'] = 1e-3
    else:
        kwargs['solver'] = solver
        kwargs['class_weight'] = class_weight
        kwargs['random_state'] = RANDOM_STATE

    return LogisticRegression(**kwargs)

def evaluate_predictions(y_true, y_pred):
    return {
        'accuracy': accuracy_score(y_true, y_pred),
        'macro_f1': f1_score(y_true, y_pred, average='macro'),
        'weighted_f1': f1_score(y_true, y_pred, average='weighted')
    }

lr_overview = build_logistic_regression()
lr_overview.fit(X_train_flat_scaled[:100], y_train_split[:100]) #
print(f"Logistic Regression overview:")
print(f"  Classes: {len(lr_overview.classes_)}")
print(f"  Features: {lr_overview.n_features_in_}")
print(f"  Parameters: {lr_overview.coef_.size + lr_overview.interce")
print(f"  Solver: {lr_overview.solver}")

```

```

Logistic Regression overview:
  Classes: 9
  Features: 2352
  Parameters: 21177
  Solver: lbfgs

```

Fully connected neural network

In this architecture, a series of hidden dense layers with ReLU activation are utilized to extract non-linear representations, followed by dropout regularization to improve generalization. Unlike CNNs, MLPs cannot directly process spatial dimensions; therefore, the 28×28×3 PathMNIST images are flattened into a 1D vector, resulting in an input shape of (2352,). The final output layer consists of 9 neurons, outputting the probability distribution for the 9 disease categories.

```

In [11]: keras.backend.clear_session()
         tf.random.set_seed(RANDOM_STATE)

         num_classes = len(np.unique(y_train_split))

```

```
def build_mlp_model(
    n_hidden_layers=2,
    n_hidden_neurons=256,
    activation_function="relu",
    dropout_rate=0.3,
    learning_rate=0.001,
    input_shape=None
):
    model = keras.models.Sequential()
    model.add(keras.layers.Input(shape=input_shape))

    for layer in range(n_hidden_layers):
        model.add(keras.layers.Dense(n_hidden_neurons, activation=activation_function))
        model.add(keras.layers.Dropout(dropout_rate))

    model.add(keras.layers.Dense(num_classes, activation="softmax"))
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy'],
    )
    return model

mlp_model_instance = build_mlp_model(input_shape=(X_train_flat_scaled.shape[1]))
mlp_model_instance.summary()
```

Model: "sequential"

Layer (type)	Output Shape	
dense (Dense)	(None, 256)	
dropout (Dropout)	(None, 256)	
dense_1 (Dense)	(None, 256)	
dropout_1 (Dropout)	(None, 256)	
dense_2 (Dense)	(None, 9)	

Total params: 670,473 (2.56 MB)

Trainable params: 670,473 (2.56 MB)

Non-trainable params: 0 (0.00 B)

Convolutional neural network

In this CNN, two convolutional layers are used to extract local image features, followed by max pooling to reduce the spatial dimensions, and finally a dense layer for classification. Since PathMNIST consists of 28×28 color images, the input shape is (28, 28, 3). The output layer has 9 neurons, corresponding to the 9 categories.

```
In [13]: num_classes = len(np.unique(y_train_split))
```

```

input_shape = X_train_img.shape[1:]

def build_cnn_model(
    conv_filters=32,
    dense_units=128,
    dropout_rate=0.3,
    learning_rate=0.001
):
    model = keras.Sequential([
        layers.Input(shape=input_shape),

        layers.Conv2D(conv_filters, kernel_size=(3, 3), activation=
layers.MaxPooling2D(pool_size=(2, 2)),

        layers.Conv2D(conv_filters * 2, kernel_size=(3, 3), activat
layers.MaxPooling2D(pool_size=(2, 2)),

        layers.Flatten(),
        layers.Dense(dense_units, activation='relu'),
        layers.Dropout(dropout_rate),
        layers.Dense(num_classes, activation='softmax')
    ], name="cnn_model")

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=learning_rate
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    return model

cnn_model = build_cnn_model()
cnn_model.summary()

```

Model: "cnn_model"

Layer (type)	Output Shape	
conv2d_2 (Conv2D)	(None, 28, 28, 32)	
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 32)	
conv2d_3 (Conv2D)	(None, 14, 14, 64)	
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 64)	
flatten_1 (Flatten)	(None, 3136)	
dense_5 (Dense)	(None, 128)	
dropout_3 (Dropout)	(None, 128)	
dense_6 (Dense)	(None, 9)	

Total params: 422,089 (1.61 MB)

Trainable params: 422,089 (1.61 MB)

Non-trainable params: 0 (0.00 B)

3. Hyperparameter tuning

Logistic Regression (Algorithm of choice from first six weeks of course)

The cell below performs a grid search over 11 values of the regularization strength `C` (log-spaced from 10^{-5} to 1) to find the optimal logistic regression hyperparameter. When CuPy is available, data arrays are transferred to GPU memory via `cp.asarray` to accelerate both training and prediction. For each `C`, the model is trained on the search split and evaluated on the held-out validation set, recording accuracy, macro F1, weighted F1, fit time, and predict time. Results are then sorted by accuracy (descending), macro F1 (descending), and fit time (ascending) to break ties, and the best `C` is extracted for later use in the final model. A horizontal bar chart visualizes validation accuracy across all 11 settings, making it easy to spot the trend in regularization strength versus performance.

```
In [14]: # list of C for tuning
lr_param_grid = list(ParameterGrid({
    'C': [1e-5, 5e-5, 1e-4, 5e-4, 1e-3, 5e-3, 1e-2, 5e-2, 1e-1, 5e-
}))

print(f'Starting logistic regression tuning over {len(lr_param_grid)}
print(f'Using GPU acceleration: {USE_CUML}')
tuning_start = time.perf_counter()

# detect GPU for acceleration, fallback to CPU if not available
if HAS_CUPY:
    X_train_gpu = cp.asarray(X_train_flat_scaled)
    y_train_gpu = cp.asarray(y_train_split)
    X_val_gpu = cp.asarray(X_val_flat_scaled)
    y_val_gpu = cp.asarray(y_val)
    print("GPU acceleration enabled")
else:
    X_train_gpu = X_train_flat_scaled
    y_train_gpu = y_train_split
    X_val_gpu = X_val_flat_scaled
    y_val_gpu = y_val

lr_results = []
for i, params in enumerate(lr_param_grid, start=1):
    print(f'[{i}/{len(lr_param_grid)}] Training with {params}...',
    model = build_logistic_regression(**params)
    fit_start = time.perf_counter()
    model.fit(X_train_gpu, y_train_gpu)
    fit_time = time.perf_counter() - fit_start
```

```

predict_start = time.perf_counter()
val_pred = model.predict(X_val_gpu)
if HAS_CUPY:
    val_pred = cp.asnumpy(val_pred)
predict_time = time.perf_counter() - predict_start

metrics = evaluate_predictions(y_val, val_pred)
lr_results.append({
    **params,
    **metrics,
    'fit_time_sec': fit_time,
    'predict_time_sec': predict_time,
})

print(
    f'    done: acc={metrics["accuracy"]:.4f}, macro_f1={metrics["macro_f1"]:.4f},
    f'fit={fit_time:.2f}s, predict={predict_time:.2f}s',
    flush=True,
)

lr_results_df = pd.DataFrame(lr_results).sort_values(
    ['accuracy', 'macro_f1', 'fit_time_sec'],
    ascending=[False, False, True],
).reset_index(drop=True)
best_lr_params = {
    'C': lr_results_df.loc[0, 'C'],
}

print(f'Logistic regression tuning finished in {(time.perf_counter() - start_time):.2f}s')
print('Logistic regression tuning results:')
print(lr_results_df.to_string(index=False))
print(f'Best logistic regression parameters: {best_lr_params}')

fig, ax = plt.subplots(figsize=(10, 4))
bar_labels = [f"C={row.C}" for row in lr_results_df.itertuples()]
ax.barh(bar_labels, lr_results_df['accuracy'], color='slateblue')
ax.set_xlabel('Validation accuracy')
ax.set_title('Logistic regression validation accuracy by hyperparameter C')
ax.invert_yaxis()
plt.tight_layout()
plt.show()

lr_results_df

```

Starting logistic regression tuning over 11 combinations

Using GPU acceleration: False

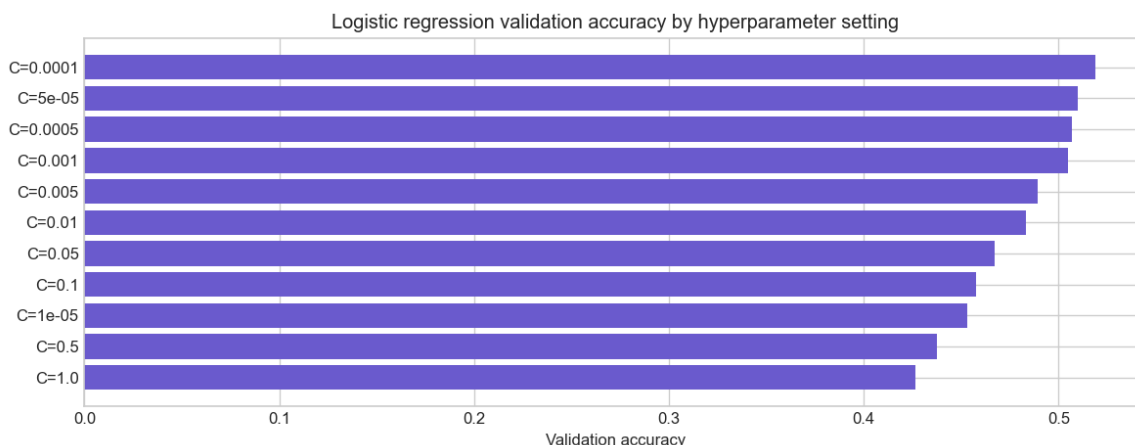
```
[1/11] Training with {'C': 1e-05}...
      done: acc=0.4531, macro_f1=0.3561, fit=16.11s, predict=0.04s
[2/11] Training with {'C': 5e-05}...
      done: acc=0.5098, macro_f1=0.4372, fit=18.26s, predict=0.02s
[3/11] Training with {'C': 0.0001}...
      done: acc=0.5191, macro_f1=0.4564, fit=25.67s, predict=0.03s
[4/11] Training with {'C': 0.0005}...
      done: acc=0.5070, macro_f1=0.4671, fit=30.01s, predict=0.02s
[5/11] Training with {'C': 0.001}...
      done: acc=0.5050, macro_f1=0.4732, fit=32.88s, predict=0.02s
[6/11] Training with {'C': 0.005}...
      done: acc=0.4891, macro_f1=0.4663, fit=40.88s, predict=0.03s
[7/11] Training with {'C': 0.01}...
      done: acc=0.4831, macro_f1=0.4633, fit=40.09s, predict=0.03s
[8/11] Training with {'C': 0.05}...
      done: acc=0.4670, macro_f1=0.4507, fit=59.68s, predict=0.03s
[9/11] Training with {'C': 0.1}...
      done: acc=0.4578, macro_f1=0.4421, fit=75.08s, predict=0.02s
[10/11] Training with {'C': 0.5}...
      done: acc=0.4373, macro_f1=0.4241, fit=145.27s, predict=0.04s
[11/11] Training with {'C': 1}...
      done: acc=0.4264, macro_f1=0.4142, fit=185.45s, predict=0.03s
```

Logistic regression tuning finished in 11.16 mins

Logistic regression tuning results:

	C	accuracy	macro_f1	weighted_f1	fit_time_sec	predict_time
0.00010	0.519062	0.456361	0.474059	25.671576	0.02	
9204						
0.00005	0.509844	0.437240	0.455607	18.260063	0.02	
3229						
0.00050	0.507031	0.467116	0.480531	30.008847	0.02	
4564						
0.00100	0.505000	0.473187	0.484736	32.878756	0.02	
3664						
0.00500	0.489063	0.466252	0.476330	40.875563	0.02	
5487						
0.01000	0.483125	0.463255	0.472671	40.093803	0.02	
5690						
0.05000	0.467031	0.450726	0.459849	59.684040	0.02	
7906						
0.10000	0.457813	0.442148	0.451130	75.082160	0.02	
4367						
0.00001	0.453125	0.356095	0.372228	16.114449	0.03	
6449						
0.50000	0.437344	0.424097	0.433196	145.273990	0.03	
9098						
1.00000	0.426406	0.414195	0.423086	185.449516	0.03	
4982						

Best logistic regression parameters: {'C': np.float64(0.0001)}



Out[14]:

	C	accuracy	macro_f1	weighted_f1	fit_time_sec	predict_time_s
0	0.00010	0.519062	0.456361	0.474059	25.671576	0.02921
1	0.00005	0.509844	0.437240	0.455607	18.260063	0.02321
2	0.00050	0.507031	0.467116	0.480531	30.008847	0.02451
3	0.00100	0.505000	0.473187	0.484736	32.878756	0.02361
4	0.00500	0.489063	0.466252	0.476330	40.875563	0.02541
5	0.01000	0.483125	0.463255	0.472671	40.093803	0.02561
6	0.05000	0.467031	0.450726	0.459849	59.684040	0.02791
7	0.10000	0.457813	0.442148	0.451130	75.082160	0.02431
8	0.00001	0.453125	0.356095	0.372228	16.114449	0.03641
9	0.50000	0.437344	0.424097	0.433196	145.273990	0.03901
10	1.00000	0.426406	0.414195	0.423086	185.449516	0.03491

Fully connected neural network

Multi-layer perceptron

In accordance with the assignment requirements, four hyperparameters are selected for an exhaustive grid search to optimise the MLP's performance on PathMNIST.

- **Network depth (n_hidden_layers: 1, 2)** — number of hidden Dense+Dropout blocks. Deeper networks can model more complex non-linear relationships, but also have more parameters and a higher risk of overfitting on a 28×28 dataset.
- **Hidden width (n_hidden_neurons: 64, 128, 256)** — number of units in each hidden layer. Controls the per-layer capacity and is the dominant driver of parameter count.
- **Regularisation (dropout_rate: 0.3, 0.4, 0.5)** — probability of zeroing each hidden activation during training. Higher dropout fights overfitting at

the cost of slower convergence and slightly higher bias.

- **Adam learning rate (optimizer__learning_rate: 0.001, 0.0005, 0.0001)** — step size of the Adam optimiser. Controls the speed/stability trade-off; smaller values converge more reliably but require more epochs.

The full grid contains $2 \times 3 \times 3 \times 3 = 54$ **unique combinations**, evaluated using a held-out validation set (the same approach used for CNN tuning). The flattened features are standardised with search_scaler (fit only on the training subset) so the validation fold remains a clean hold-out.

```
In [15]: # MLP tuning with KerasTuner – reuses build_mlp_model
def build_mlp_tunable(hp):
    return build_mlp_model(
        n_hidden_layers=hp.Choice('n_hidden_layers', [1, 2]),
        n_hidden_neurons=hp.Choice('n_hidden_neurons', [64, 128, 256]),
        dropout_rate=hp.Choice('dropout_rate', [0.3, 0.4, 0.5]),
        learning_rate=hp.Choice('learning_rate', [0.001, 0.0005, 0.0001]),
        input_shape=(X_train_flat_scaled.shape[1],),
    )

# grid search over 2x3x3x3 = 54 combinations of MLP hyperparameters
mlp_tuner = kt.GridSearch(
    build_mlp_tunable,
    objective='val_accuracy',
    max_trials=54,
    directory='./kt_models/mlp_tuning',
    project_name='mlp_grid'
)

print("Starting KerasTuner GridSearch over 54 combinations (2x3x3x3)
start_time = time.time()

mlp_tuner.search(
    X_train_flat_scaled, y_train_split,
    validation_data=(X_val_flat_scaled, y_val),
    epochs=10, batch_size=64, verbose=1
)

print(f"\nGrid search complete, overall time used: {(time.time() - start_time):.2f} mins")
```

Reloading Tuner from ./kt_models/mlp_tuning/mlp_grid/tuner0.json
Starting KerasTuner GridSearch over 54 combinations (2x3x3x3)...

Grid search complete, overall time used: 0.00 mins

```
In [16]: # result dataframe for MLP tuning
mlp_trials = mlp_tuner.oracle.get_best_trials(num_trials=54)

mlp_tuning_results = []
for trial in mlp_trials:
    mlp_tuning_results.append({
        'n_hidden_layers': trial.hyperparameters.values['n_hidden_layers'],
        'n_hidden_neurons': trial.hyperparameters.values['n_hidden_neurons'],
        'dropout_rate': trial.hyperparameters.values['dropout_rate']
    })
```

```

        'learning_rate': trial.hyperparameters.values['learning_rate'],
        'val_accuracy': trial.score,
    })

mlp_tuning_results_df = pd.DataFrame(mlp_tuning_results).sort_values(
    'val_accuracy', ascending=False
).reset_index(drop=True)

display_cols = ['n_hidden_layers', 'n_hidden_neurons',
                'dropout_rate', 'learning_rate', 'val_accuracy']

print("\n--- Top 10 combinations ---")
display(mlp_tuning_results_df[display_cols].head(10))

```

--- Top 10 combinations ---

	n_hidden_layers	n_hidden_neurons	dropout_rate	learning_rate	val_accuracy
0	2	256	0.3	0.0005	0.637
1	2	256	0.3	0.0010	0.630
2	2	256	0.3	0.0001	0.627
3	2	256	0.4	0.0005	0.625
4	2	128	0.3	0.0005	0.624
5	1	256	0.5	0.0005	0.622
6	2	128	0.3	0.0010	0.622
7	1	256	0.4	0.0005	0.622
8	1	256	0.3	0.0005	0.618
9	1	256	0.3	0.0001	0.616

```

In [17]: best_mlp_hps = mlp_tuner.get_best_hyperparameters()[0]
best_mlp_score = mlp_tuner.oracle.get_best_trials(1)[0].score

print(f"Best validation accuracy: {best_mlp_score:.4f}")
print("The optimal combination of hyperparameters found:")
for param, value in best_mlp_hps.values.items():
    print(f" {param}: {value}")

```

Best validation accuracy: 0.6378

The optimal combination of hyperparameters found:

```

n_hidden_layers: 2
n_hidden_neurons: 256
dropout_rate: 0.3
learning_rate: 0.0005

```

```

In [18]: # visualize MLP tuning results, combinations vs validation accuracy
fig, ax = plt.subplots(figsize=(10, 5))

ax.plot(
    range(len(mlp_tuning_results_df)),
    mlp_tuning_results_df['val_accuracy'],

```

```

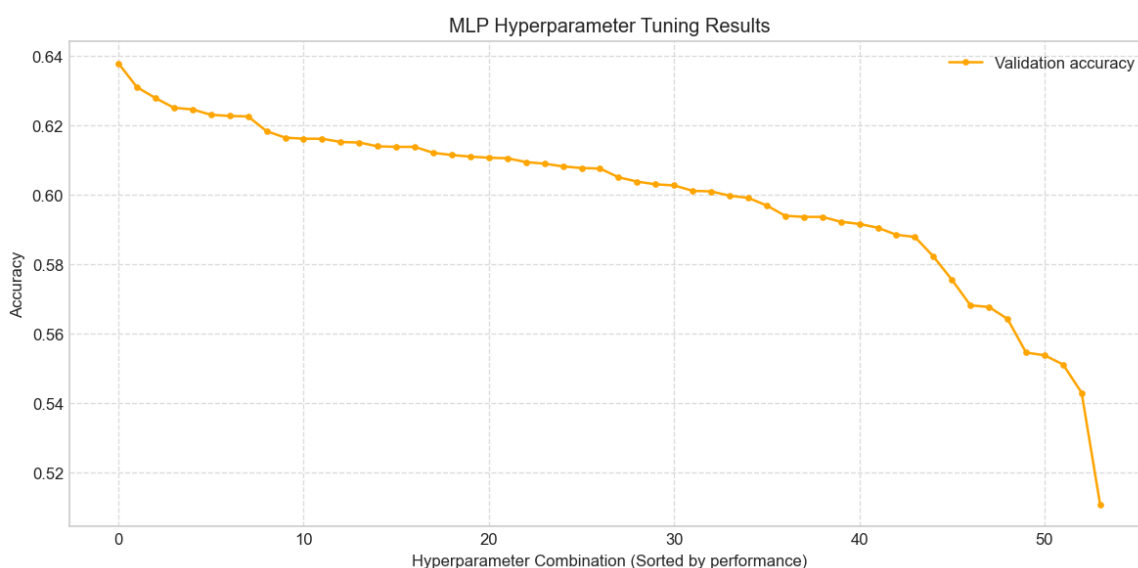
marker='.',
linestyle='--',
color='orange',
label='Validation accuracy'
)

ax.set_title('MLP Hyperparameter Tuning Results')
ax.set_xlabel('Hyperparameter Combination (Sorted by performance)')
ax.set_ylabel('Accuracy')
ax.legend()

ax.grid(True, linestyle='--', alpha=0.7)

fig.tight_layout()
fig.savefig(output_dir / 'mlp_hyperparameter_tuning.png', bbox_inches=
plt.show()

```



Convolutional neural network

In accordance with the assignment requirements, four hyperparameters were selected for an exhaustive grid search to optimise the CNN model on the PathMNIST image classification task.

Convolution filters (`conv_filters` : 32, 64) — the number of filters in the first convolutional layer. The second convolutional layer uses twice this number of filters. Increasing the number of filters allows the CNN to learn more local image patterns and texture features, but also increases the number of parameters and runtime.

Dense layer size (`dense_units` : 64, 128, 256) — the number of neurons in the fully connected layer after the convolution and pooling layers. This controls the capacity of the final classification part of the model. Larger dense layers can learn more complex feature combinations, but may also increase the risk of overfitting.

Regularisation (`dropout_rate` : 0.3, 0.4, 0.5) — the proportion of dense-layer activations randomly set to zero during training. Dropout is used to reduce overfitting. A higher dropout rate provides stronger regularisation, but may also make learning slower or cause underfitting if too much information is removed.

Adam learning rate (`learning_rate` : 0.001, 0.0005, 0.0001) — the initial step size used by the Adam optimiser. A larger learning rate can train faster but may be less stable, while a smaller learning rate can be more stable but may require more epochs to converge.

The full grid contains $2 \times 3 \times 3 \times 3 = 54$ unique hyperparameter combinations. Each combination is trained on the training subset and evaluated using the held-out validation set. The validation accuracy is used as the selection criterion because the test set should only be used for the final model comparison.

```
In [19]: # CNN tuning with KerasTuner – reuses build_cnn_model
def build_cnn_tunable(hp):
    return build_cnn_model(
        conv_filters=hp.Choice('conv_filters', [32, 64]),
        dense_units=hp.Choice('dense_units', [64, 128, 256]),
        dropout_rate=hp.Choice('dropout_rate', [0.3, 0.4, 0.5]),
        learning_rate=hp.Choice('learning_rate', [0.001, 0.0005, 0.0001])
    )

# grid search over 2x3x3x3 = 54 combinations of CNN hyperparameters
cnn_tuner = kt.GridSearch(
    build_cnn_tunable,
    objective='val_accuracy',
    max_trials=54,
    directory='./kt_models/cnn_tuning',
    project_name='cnn_grid'
)

print("Starting KerasTuner GridSearch over 54 combinations (2x3x3x3)
start_time = time.time()

reduce_lr = keras.callbacks.ReduceLROnPlateau(
    monitor='val_accuracy', factor=0.5, patience=3, min_lr=1e-6, verbose=1
)

cnn_tuner.search(
    X_train_img, y_train_split,
    validation_data=(X_val_img, y_val),
    epochs=10, batch_size=128, verbose=1,
    callbacks=[reduce_lr]
)

print(f"\nGrid search complete, overall time used: {(time.time() - start_time):.2f} seconds")
```

Reloading Tuner from ./kt_models/cnn_tuning/cnn_grid/tuner0.json
Starting KerasTuner GridSearch over 54 combinations (2×3×3×3)...

Grid search complete, overall time used: 0.00 mins

Then select the optimal parameters and save the results.

```
In [20]: # result dataframe for CNN tuning
cnn_trials = cnn_tuner.oracle.get_best_trials(num_trials=54)

cnn_tuning_results = []
for trial in cnn_trials:
    cnn_tuning_results.append({
        'conv_filters': trial.hyperparameters.values['conv_filters'],
        'dense_units': trial.hyperparameters.values['dense_units'],
        'dropout_rate': trial.hyperparameters.values['dropout_rate'],
        'learning_rate': trial.hyperparameters.values['learning_rate'],
        'val_accuracy': trial.score,
    })

cnn_tuning_results_df = pd.DataFrame(cnn_tuning_results).sort_values(
    'val_accuracy', ascending=False
).reset_index(drop=True)

display_cols = ['conv_filters', 'dense_units',
                'dropout_rate', 'learning_rate', 'val_accuracy']

print("\n--- Top 10 combinations ---")
display(cnn_tuning_results_df[display_cols].head(10))
```

--- Top 10 combinations ---

	conv_filters	dense_units	dropout_rate	learning_rate	val_accuracy
0	64	128	0.3	0.0010	0.800469
1	32	256	0.4	0.0010	0.798281
2	64	256	0.3	0.0010	0.796719
3	64	256	0.4	0.0010	0.796406
4	64	256	0.4	0.0005	0.795781
5	64	128	0.4	0.0005	0.794531
6	64	256	0.5	0.0010	0.793906
7	64	256	0.3	0.0005	0.790000
8	32	256	0.4	0.0005	0.788750
9	64	128	0.3	0.0005	0.788594

```
In [21]: # Automatically select the set of parameters with the highest valid
best_cnn_hps = cnn_tuner.get_best_hyperparameters()[0]
best_cnn_score = cnn_tuner.oracle.get_best_trials(1)[0].score

print(f"Best validation accuracy: {best_cnn_score:.4f}")
```

```
print("The optimal combination of hyperparameters found:")
for param, value in best_cnn_hps.values.items():
    print(f" {param}: {value}")
```

Best validation accuracy: 0.8005

The optimal combination of hyperparameters found:

```
conv_filters: 64
dense_units: 128
dropout_rate: 0.3
learning_rate: 0.001
```

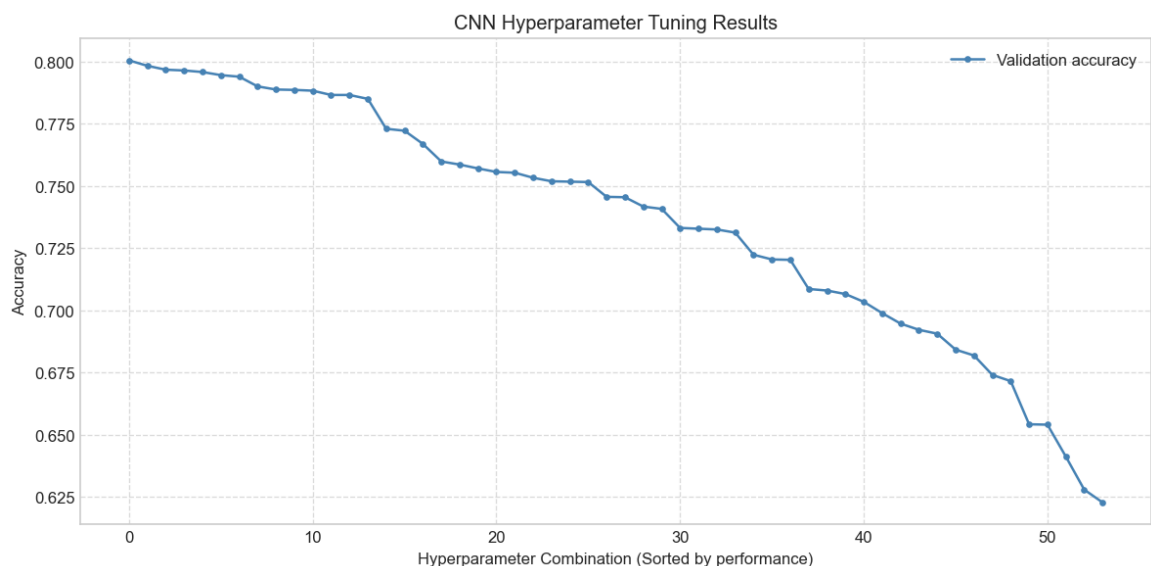
```
In [22]: # visualize CNN tuning results, sorted by performance
fig, ax = plt.subplots(figsize=(10, 5))

ax.plot(
    range(len(cnn_tuning_results_df)),
    cnn_tuning_results_df['val_accuracy'],
    marker='.',
    linestyle='--',
    color='steelblue',
    label='Validation accuracy'
)

ax.set_title('CNN Hyperparameter Tuning Results')
ax.set_xlabel('Hyperparameter Combination (Sorted by performance)')
ax.set_ylabel('Accuracy')
ax.legend()

ax.grid(True, linestyle='--', alpha=0.7)

fig.tight_layout()
fig.savefig(output_dir / 'cnn_hyperparameter_tuning.png', bbox_inches=
plt.show()
```



4. Final models

In this section, please ensure to include cells to train each model with its best hyperparameter combination independently of the hyperparameter tuning cells,

i.e. don't rely on the hyperparameter tuning cells having been run.

Logistic Regression (Algorithm of choice from first six weeks of course)

The cell below trains the final logistic regression model using the best C found during tuning. It fits on the full training set, evaluates on the test set, and reports accuracy, macro F1, weighted F1, parameter count, and runtime — all with GPU acceleration via CuPy when available.

```
In [23]: # use best hyperparameters from tuning
try:
    lr_final_params = best_lr_params
    print(f"Using best params from tuning: {lr_final_params}")
except NameError:
    lr_final_params = {'C': 0.0001}
    print(f"Tuning results not found, using fallback: {lr_final_params}")

final_lr_model = build_logistic_regression(**lr_final_params)

# use GPU acceleration for final training, fallback to CPU if not available
if HAS_CUPY:
    X_train_full_gpu = cp.asarray(X_train_full_flat_scaled)
    y_train_gpu = cp.asarray(y_train)
    X_test_full_gpu = cp.asarray(X_test_full_flat_scaled)
else:
    X_train_full_gpu = X_train_full_flat_scaled
    y_train_gpu = y_train
    X_test_full_gpu = X_test_full_flat_scaled

fit_start = time.perf_counter()
final_lr_model.fit(X_train_full_gpu, y_train_gpu)
lr_fit_time = time.perf_counter() - fit_start

predict_start = time.perf_counter()
lr_test_pred = final_lr_model.predict(X_test_full_gpu)
if HAS_CUPY:
    lr_test_pred = cp.asnumpy(lr_test_pred)
lr_predict_time = time.perf_counter() - predict_start

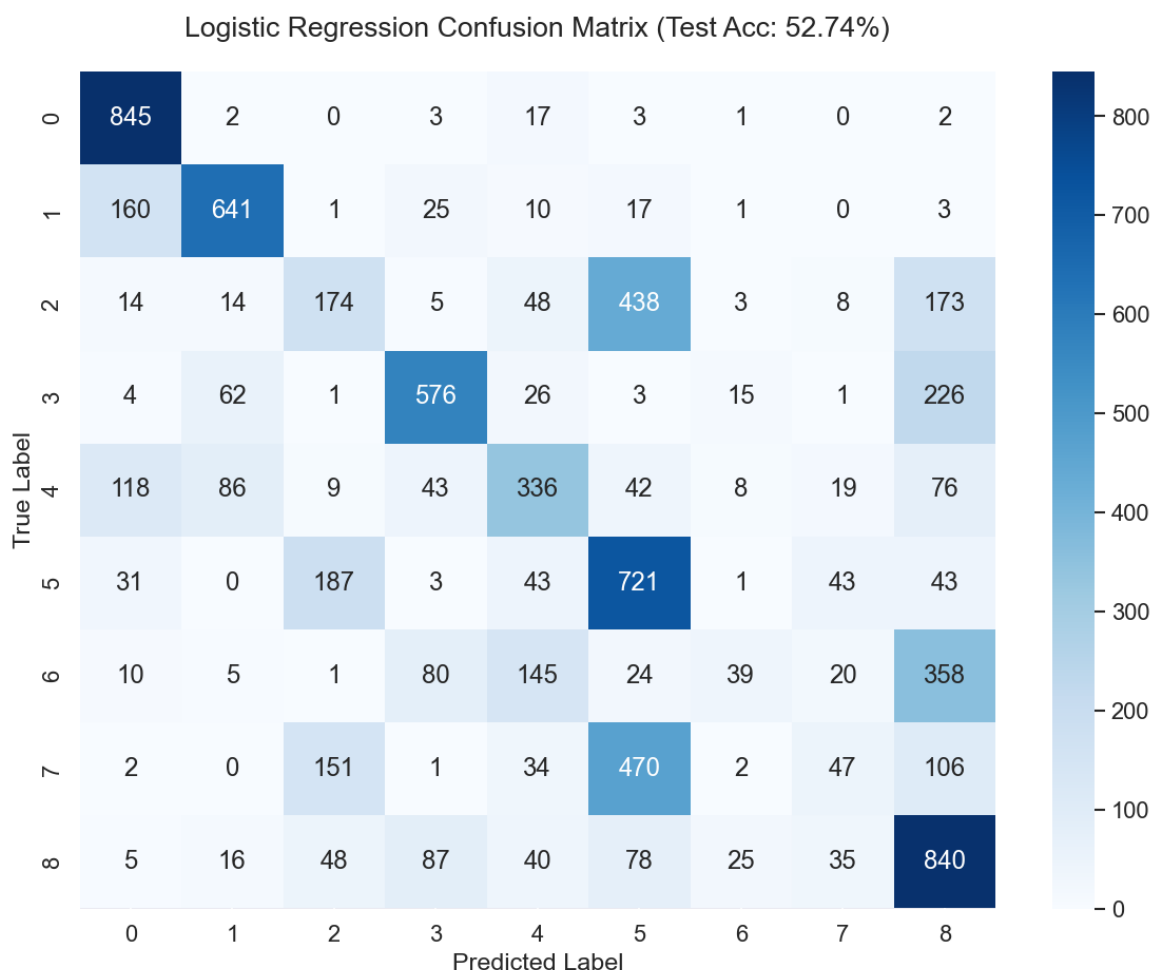
lr_test_metrics = evaluate_predictions(y_test, lr_test_pred)
lr_final_summary = {
    'model': 'Logistic regression',
    'best_hyperparameters': f"C={lr_final_params['C']}",
    'params': int(final_lr_model.coef_.size + final_lr_model.intercept_.size),
    **lr_test_metrics,
    'fit_time_sec': lr_fit_time,
    'predict_time_sec': lr_predict_time,
}

print('Final logistic regression test results:')
print(lr_final_summary)
```

```
Using best params from tuning: {'C': np.float64(0.0001)}  
Final logistic regression test results:  
{'model': 'Logistic regression', 'best_hyperparameters': 'C=0.0001',  
'params': 21177, 'accuracy': 0.527375, 'macro_f1': 0.467838167907342  
3, 'weighted_f1': 0.4840699075095356, 'fit_time_sec': 38.40192445879  
802, 'predict_time_sec': 0.04764824965968728}
```

```
In [24]: # Confusion matrix  
cm = confusion_matrix(y_test, lr_test_pred)  
  
plt.figure(figsize=(10, 8))  
sns.set_theme(font_scale=1.2)  
  
# Drawing a heatmap  
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',  
            xticklabels=np.unique(y_test),  
            yticklabels=np.unique(y_test))  
  
plt.title(f'Logistic Regression Confusion Matrix (Test Acc: {lr_test_acc})')  
plt.ylabel('True Label', fontsize=14)  
plt.xlabel('Predicted Label', fontsize=14)  
plt.tight_layout()  
  
print("Due to random initialization and stochastic optimisation, neural network  
results may vary slightly between runs.")  
  
plt.savefig(output_dir / 'log_reg_confusion_matrix.png', dpi=300)  
plt.show()
```

Due to random initialization and stochastic optimisation, neural network results may vary slightly between runs.



Fully connected neural network

```
In [25]: keras.backend.clear_session()
keras.utils.set_random_seed(RANDOM_STATE)

try:
    best_mlp_params = dict(best_mlp_hps.values)
    print(f"Using best params from KerasTuner: {best_mlp_params}")
except NameError:
    best_mlp_params = {
        'n_hidden_layers': 1,
        'n_hidden_neurons': 256,
        'dropout_rate': 0.3,
        'learning_rate': 0.001,
    }
    print(f"Tuner results unavailable, using fallback: {best_mlp_params}")

final_mlp_model = build_mlp_model(
    n_hidden_layers=best_mlp_params['n_hidden_layers'],
    n_hidden_neurons=best_mlp_params['n_hidden_neurons'],
    dropout_rate=best_mlp_params['dropout_rate'],
    learning_rate=best_mlp_params['learning_rate'],
    input_shape=(X_train_full_flat_scaled.shape[1],),
)

# use larger epochs for final training to achieve better convergence
mlp_start_time = time.time()
```

```

final_mlp_history = final_mlp_model.fit(
    X_train_full_flat_scaled,
    y_train,
    epochs=30,
    batch_size=64,
    verbose=1,
)
final_mlp_runtime = time.time() - mlp_start_time
print(f"MLP final training runtime: {final_mlp_runtime:.2f} seconds

mlp_test_probs = final_mlp_model.predict(X_test_full_flat_scaled, v
y_pred_mlp = np.argmax(mlp_test_probs, axis=1)

test_accuracy = accuracy_score(y_test, y_pred_mlp)
print(f"\nAccuracy of MLP model on test set: {test_accuracy:.4f}")

print("\n--- Classification Report ---")
print(classification_report(y_test, y_pred_mlp, digits=4, zero_divi

```

Using best params from KerasTuner: {'n_hidden_layers': 2, 'n_hidden_neurons': 256, 'dropout_rate': 0.3, 'learning_rate': 0.0005}

Epoch 1/30

500/500 ————— 3s 4ms/step - accuracy: 0.4425 - loss: 1.4777

Epoch 2/30

500/500 ————— 2s 4ms/step - accuracy: 0.5427 - loss: 1.1954

Epoch 3/30

500/500 ————— 2s 4ms/step - accuracy: 0.5825 - loss: 1.0983

Epoch 4/30

500/500 ————— 2s 4ms/step - accuracy: 0.6000 - loss: 1.0402

Epoch 5/30

500/500 ————— 2s 4ms/step - accuracy: 0.6223 - loss: 0.9917

Epoch 6/30

500/500 ————— 2s 4ms/step - accuracy: 0.6335 - loss: 0.9600

Epoch 7/30

500/500 ————— 2s 5ms/step - accuracy: 0.6495 - loss: 0.9169

Epoch 8/30

500/500 ————— 2s 4ms/step - accuracy: 0.6615 - loss: 0.8872

Epoch 9/30

500/500 ————— 2s 4ms/step - accuracy: 0.6712 - loss: 0.8661

Epoch 10/30

500/500 ————— 2s 4ms/step - accuracy: 0.6763 - loss: 0.8418

Epoch 11/30

500/500 ————— 2s 4ms/step - accuracy: 0.6867 - loss: 0.8170

Epoch 12/30

500/500 ————— 2s 4ms/step - accuracy: 0.6913 - loss: 0.8003
Epoch 13/30
500/500 ————— 2s 4ms/step - accuracy: 0.7035 - loss: 0.7793
Epoch 14/30
500/500 ————— 2s 4ms/step - accuracy: 0.7107 - loss: 0.7556
Epoch 15/30
500/500 ————— 2s 4ms/step - accuracy: 0.7194 - loss: 0.7390
Epoch 16/30
500/500 ————— 2s 4ms/step - accuracy: 0.7293 - loss: 0.7166
Epoch 17/30
500/500 ————— 2s 4ms/step - accuracy: 0.7344 - loss: 0.7025
Epoch 18/30
500/500 ————— 2s 4ms/step - accuracy: 0.7446 - loss: 0.6792
Epoch 19/30
500/500 ————— 2s 4ms/step - accuracy: 0.7471 - loss: 0.6748
Epoch 20/30
500/500 ————— 2s 4ms/step - accuracy: 0.7520 - loss: 0.6583
Epoch 21/30
500/500 ————— 2s 4ms/step - accuracy: 0.7567 - loss: 0.6410
Epoch 22/30
500/500 ————— 2s 4ms/step - accuracy: 0.7639 - loss: 0.6317
Epoch 23/30
500/500 ————— 2s 4ms/step - accuracy: 0.7682 - loss: 0.6189
Epoch 24/30
500/500 ————— 2s 4ms/step - accuracy: 0.7772 - loss: 0.5999
Epoch 25/30
500/500 ————— 2s 4ms/step - accuracy: 0.7771 - loss: 0.5951
Epoch 26/30
500/500 ————— 2s 4ms/step - accuracy: 0.7840 - loss: 0.5766
Epoch 27/30
500/500 ————— 2s 5ms/step - accuracy: 0.7903 - loss: 0.5691
Epoch 28/30
500/500 ————— 2s 4ms/step - accuracy: 0.7922 - loss: 0.5651
Epoch 29/30
500/500 ————— 2s 4ms/step - accuracy: 0.7960 - loss: 0.5548
Epoch 30/30
500/500 ————— 2s 4ms/step - accuracy: 0.8011 - loss: 0.5446

MLP final training runtime: 63.10 seconds

Accuracy of MLP model on test set: 0.6706

--- Classification Report ---

	precision	recall	f1-score	support
0	0.8112	0.9645	0.8812	873
1	0.9553	0.8974	0.9255	858
2	0.5966	0.3979	0.4774	877
3	0.8712	0.8512	0.8611	914
4	0.6481	0.5672	0.6049	737
5	0.6713	0.6269	0.6483	1072
6	0.4343	0.4941	0.4623	682
7	0.4526	0.6162	0.5219	813
8	0.6075	0.5945	0.6009	1174
accuracy			0.6706	8000
macro avg	0.6720	0.6678	0.6648	8000
weighted avg	0.6777	0.6706	0.6694	8000

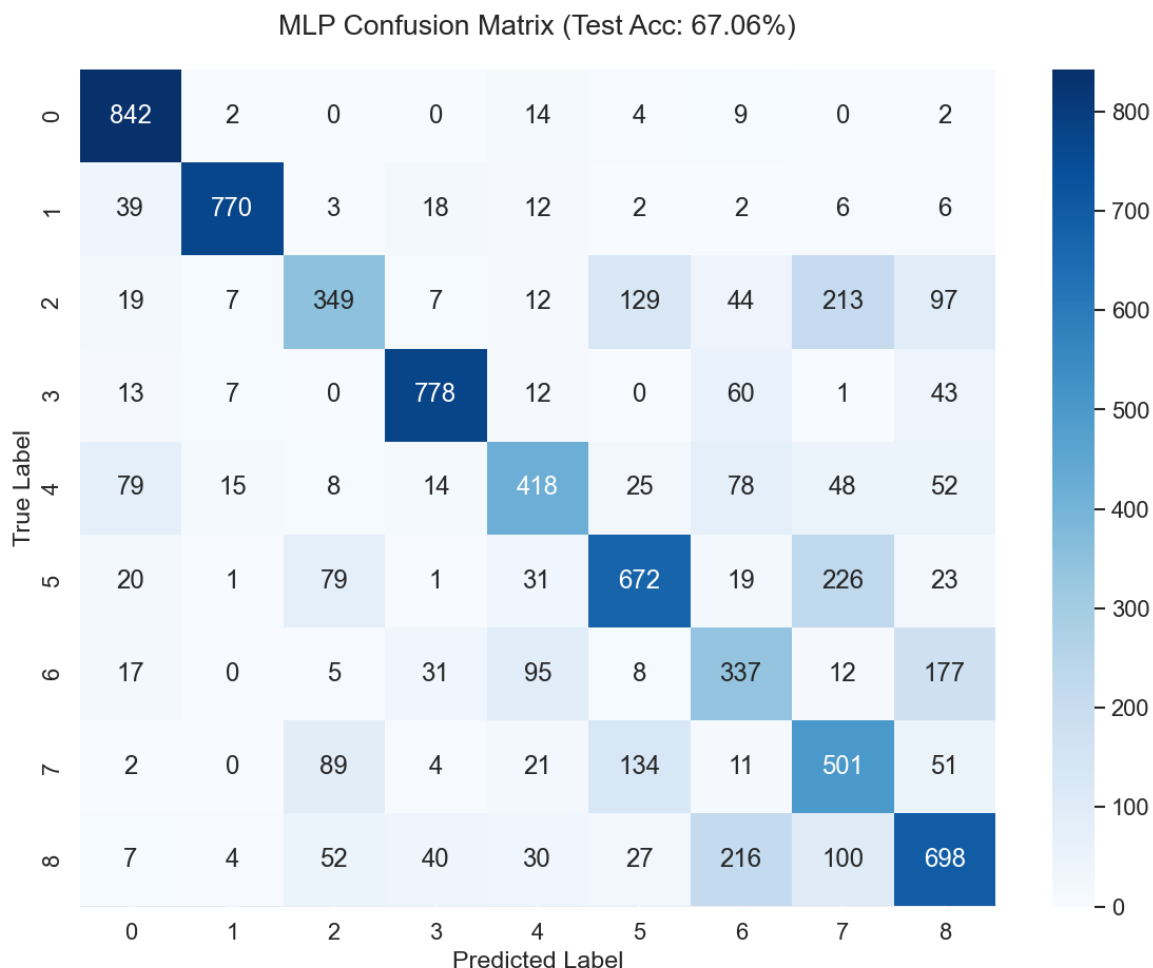
```
In [26]: # Confusion matrix
cm = confusion_matrix(y_test, y_pred_mlp)

plt.figure(figsize=(10, 8))
sns.set_theme(font_scale=1.2)

# Drawing a heatmap
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=np.unique(y_test),
            yticklabels=np.unique(y_test))

plt.title(f'MLP Confusion Matrix (Test Acc: {test_accuracy:.2%})',
plt.ylabel('True Label', fontsize=14)
plt.xlabel('Predicted Label', fontsize=14)
plt.tight_layout()

plt.savefig(output_dir / 'mlp_confusion_matrix.png', dpi=300)
plt.show()
```



Convolutional neural network

Using the optimal hyperparameter combination identified during the grid search, the final CNN model was retrained on the full training dataset to ensure a fair comparison with the MLP model. The selected configuration used 64 convolutional filters, 128 dense units, a dropout rate of 0.3, and an Adam learning rate of 0.001.

The final CNN architecture consisted of two convolutional layers with ReLU activation and max-pooling operations, followed by a fully connected dense layer and dropout regularisation. The model was trained for 30 epochs using a batch size of 128. A ReduceLROnPlateau callback was applied to automatically reduce the learning rate when the validation accuracy stopped improving.

After training, the final CNN model achieved a test accuracy of 83.51%, with a test loss of 0.4472. The macro-average F1-score was 0.8325 and the weighted-average F1-score was 0.8349. The total training runtime was approximately 640.5 seconds.

The confusion matrix and classification report indicate that the CNN model achieved strong overall classification performance, particularly for classes 0,

1, and 3, which obtained F1-scores above 0.95. However, some classes such as 2, 6, and 7 were more difficult to distinguish, suggesting that these image categories share more visually similar texture patterns.

```
In [28]: keras.backend.clear_session()
keras.utils.set_random_seed(RANDOM_STATE)

try:
    best_cnn_params = dict(best_cnn_hps.values)
    print(f"Using best params from KerasTuner: {best_cnn_params}")
except NameError:
    best_cnn_params = {
        'conv_filters': 64,
        'dense_units': 256,
        'dropout_rate': 0.3,
        'learning_rate': 0.001,
    }
    print(f"Tuner results unavailable, using fallback: {best_cnn_pa

final_cnn_model = build_cnn_model(**best_cnn_params)

start_time = time.time()

reduce_lr = keras.callbacks.ReduceLRonPlateau(
    monitor='val_accuracy', factor=0.5, patience=3, min_lr=1e-6, ve
)

final_cnn_history = final_cnn_model.fit(
    X_train_norm,
    y_train,
    validation_data=(X_val_img, y_val),
    epochs=30,
    batch_size=128,
    verbose=1,
    callbacks=[reduce_lr]
)

final_cnn_runtime = time.time() - start_time
```

Using best params from KerasTuner: {'conv_filters': 64, 'dense_units': 128, 'dropout_rate': 0.3, 'learning_rate': 0.001}

Epoch 1/30

250/250 ————— **18s** 69ms/step – accuracy: 0.3847 – loss: 1.6279 – val_accuracy: 0.4784 – val_loss: 1.3950 – learning_rate: 0.0010

Epoch 2/30

250/250 ————— **18s** 71ms/step – accuracy: 0.5778 – loss: 1.1388 – val_accuracy: 0.5650 – val_loss: 1.1641 – learning_rate: 0.0010

Epoch 3/30

250/250 ————— **17s** 68ms/step – accuracy: 0.6443 – loss: 0.9782 – val_accuracy: 0.6662 – val_loss: 0.9270 – learning_rate: 0.0010

Epoch 4/30

250/250 ————— **18s** 72ms/step – accuracy: 0.6732 – loss:

s: 0.9092 - val_accuracy: 0.7134 - val_loss: 0.8001 - learning_rate: 0.0010
Epoch 5/30
250/250  18s 73ms/step - accuracy: 0.6976 - loss: 0.8395 - val_accuracy: 0.7525 - val_loss: 0.7066 - learning_rate: 0.0010
Epoch 6/30
250/250  19s 75ms/step - accuracy: 0.7152 - loss: 0.7867 - val_accuracy: 0.7667 - val_loss: 0.6677 - learning_rate: 0.0010
Epoch 7/30
250/250  21s 85ms/step - accuracy: 0.7295 - loss: 0.7466 - val_accuracy: 0.7588 - val_loss: 0.6681 - learning_rate: 0.0010
Epoch 8/30
250/250  21s 83ms/step - accuracy: 0.7423 - loss: 0.7103 - val_accuracy: 0.7675 - val_loss: 0.6434 - learning_rate: 0.0010
Epoch 9/30
250/250  20s 78ms/step - accuracy: 0.7431 - loss: 0.7078 - val_accuracy: 0.7739 - val_loss: 0.6205 - learning_rate: 0.0010
Epoch 10/30
250/250  22s 87ms/step - accuracy: 0.7538 - loss: 0.6804 - val_accuracy: 0.7939 - val_loss: 0.5589 - learning_rate: 0.0010
Epoch 11/30
250/250  22s 86ms/step - accuracy: 0.7655 - loss: 0.6494 - val_accuracy: 0.8042 - val_loss: 0.5331 - learning_rate: 0.0010
Epoch 12/30
250/250  22s 86ms/step - accuracy: 0.7755 - loss: 0.6177 - val_accuracy: 0.8105 - val_loss: 0.5095 - learning_rate: 0.0010
Epoch 13/30
250/250  24s 95ms/step - accuracy: 0.7791 - loss: 0.6033 - val_accuracy: 0.8191 - val_loss: 0.4845 - learning_rate: 0.0010
Epoch 14/30
250/250  24s 94ms/step - accuracy: 0.7849 - loss: 0.5869 - val_accuracy: 0.8259 - val_loss: 0.4737 - learning_rate: 0.0010
Epoch 15/30
250/250  21s 85ms/step - accuracy: 0.7946 - loss: 0.5624 - val_accuracy: 0.8177 - val_loss: 0.4804 - learning_rate: 0.0010
Epoch 16/30
250/250  22s 89ms/step - accuracy: 0.7974 - loss: 0.5473 - val_accuracy: 0.8455 - val_loss: 0.4089 - learning_rate: 0.0010
Epoch 17/30
250/250  23s 91ms/step - accuracy: 0.8012 - loss: 0.5427 - val_accuracy: 0.8323 - val_loss: 0.4601 - learning_rate: 0.0010
Epoch 18/30
250/250  24s 94ms/step - accuracy: 0.8113 - loss:

```

s: 0.5084 - val_accuracy: 0.8470 - val_loss: 0.4142 - learning_rate:
0.0010
Epoch 19/30
250/250 ————— 22s 90ms/step - accuracy: 0.8130 - los
s: 0.5062 - val_accuracy: 0.8047 - val_loss: 0.5211 - learning_rate:
0.0010
Epoch 20/30
250/250 ————— 23s 91ms/step - accuracy: 0.8243 - los
s: 0.4812 - val_accuracy: 0.8547 - val_loss: 0.3841 - learning_rate:
0.0010
Epoch 21/30
250/250 ————— 24s 94ms/step - accuracy: 0.8205 - los
s: 0.4809 - val_accuracy: 0.8578 - val_loss: 0.3876 - learning_rate:
0.0010
Epoch 22/30
250/250 ————— 24s 96ms/step - accuracy: 0.8305 - los
s: 0.4572 - val_accuracy: 0.8595 - val_loss: 0.3818 - learning_rate:
0.0010
Epoch 23/30
250/250 ————— 22s 88ms/step - accuracy: 0.8295 - los
s: 0.4589 - val_accuracy: 0.8619 - val_loss: 0.3671 - learning_rate:
0.0010
Epoch 24/30
250/250 ————— 22s 89ms/step - accuracy: 0.8328 - los
s: 0.4485 - val_accuracy: 0.8656 - val_loss: 0.3596 - learning_rate:
0.0010
Epoch 25/30
250/250 ————— 22s 88ms/step - accuracy: 0.8422 - los
s: 0.4268 - val_accuracy: 0.8467 - val_loss: 0.4123 - learning_rate:
0.0010
Epoch 26/30
250/250 ————— 22s 89ms/step - accuracy: 0.8456 - los
s: 0.4139 - val_accuracy: 0.8878 - val_loss: 0.3035 - learning_rate:
0.0010
Epoch 27/30
250/250 ————— 22s 87ms/step - accuracy: 0.8539 - los
s: 0.3958 - val_accuracy: 0.8994 - val_loss: 0.2770 - learning_rate:
0.0010
Epoch 28/30
250/250 ————— 22s 88ms/step - accuracy: 0.8572 - los
s: 0.3847 - val_accuracy: 0.8914 - val_loss: 0.2913 - learning_rate:
0.0010
Epoch 29/30
250/250 ————— 22s 89ms/step - accuracy: 0.8657 - los
s: 0.3603 - val_accuracy: 0.8944 - val_loss: 0.2797 - learning_rate:
0.0010
Epoch 30/30
250/250 ————— 0s 83ms/step - accuracy: 0.8620 - loss:
0.3639
Epoch 30: ReduceLR0nPlateau reducing learning rate to 0.000500000023
7487257.
250/250 ————— 22s 89ms/step - accuracy: 0.8624 - los
s: 0.3637 - val_accuracy: 0.8928 - val_loss: 0.2820 - learning_rate:
0.0010

```

In [29]: *# Test Set Evaluation*

```

cnn_test_loss, cnn_test_accuracy = final_cnn_model.evaluate(
    X_test_norm,
    y_test,
    verbose=0
)

cnn_test_probabilities = final_cnn_model.predict(X_test_norm)
cnn_test_predictions = np.argmax(cnn_test_probabilities, axis=1)

print(f'CNN test accuracy: {cnn_test_accuracy:.4f}')
print(f'CNN test loss: {cnn_test_loss:.4f}')
print(f'CNN final training runtime: {final_cnn_runtime:.2f} seconds

```

250/250 ————— 2s 7ms/step
 CNN test accuracy: 0.8351
 CNN test loss: 0.4705
 CNN final training runtime: 640.52 seconds

```

In [30]: # Print Categorized Report
print(classification_report(y_test, cnn_test_predictions, zero_divi

```

	precision	recall	f1-score	support
0	0.95	0.98	0.96	873
1	0.98	0.97	0.97	858
2	0.72	0.73	0.73	877
3	0.88	0.99	0.93	914
4	0.87	0.80	0.83	737
5	0.87	0.72	0.79	1072
6	0.74	0.78	0.76	682
7	0.65	0.71	0.68	813
8	0.86	0.83	0.84	1174
accuracy			0.84	8000
macro avg	0.83	0.83	0.83	8000
weighted avg	0.84	0.84	0.83	8000

```

In [31]: # Confusion matrix
cm = confusion_matrix(y_test, cnn_test_predictions)

plt.figure(figsize=(10, 8))
sns.set_theme(font_scale=1.2)

# Drawing a heatmap
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=np.unique(y_test),
            yticklabels=np.unique(y_test))

plt.title(f'CNN Confusion Matrix (Test Acc: {cnn_test_accuracy:.2%})
plt.ylabel('True Label', fontsize=14)
plt.xlabel('Predicted Label', fontsize=14)
plt.tight_layout()

print("Due to random initialization and stochastic mini-batch train
plt.savefig(output_dir / 'cnn_confusion_matrix.png', dpi=300)

```

```
plt.show()
```

Due to random initialization and stochastic mini-batch training, the final test accuracy may vary slightly between runs.

CNN Confusion Matrix (Test Acc: 83.51%)

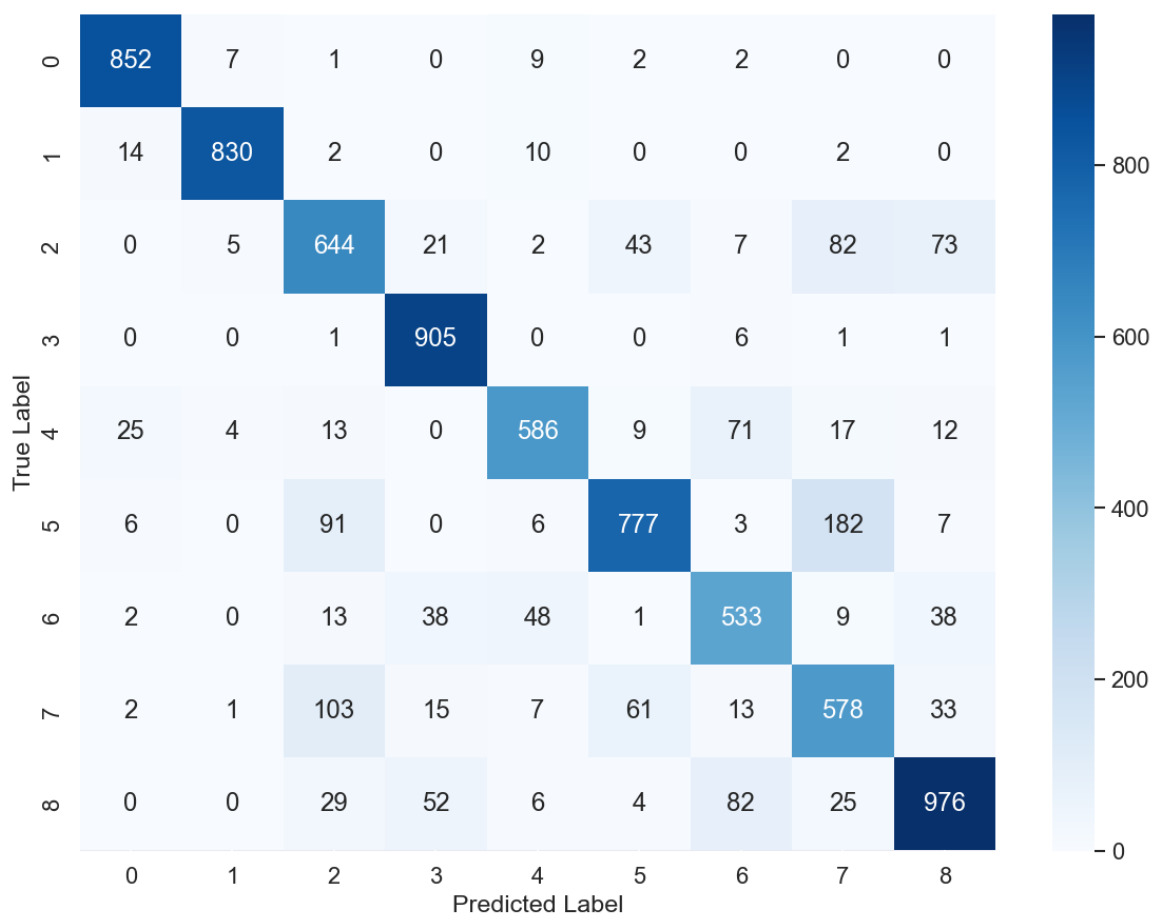


Table for final comparison of the three models

```
In [37]: # Final model comparison table

lr_macro_f1 = f1_score(y_test, lr_test_pred, average='macro')
lr_weighted_f1 = f1_score(y_test, lr_test_pred, average='weighted')

mlp_macro_f1 = f1_score(y_test, y_pred_mlp, average='macro')
mlp_weighted_f1 = f1_score(y_test, y_pred_mlp, average='weighted')

cnn_macro_f1 = f1_score(y_test, cnn_test_predictions, average='macro')
cnn_weighted_f1 = f1_score(y_test, cnn_test_predictions, average='weighted')

model_comparison_df = pd.DataFrame([
    {
        'Model': 'Logistic Regression',
        'Best hyperparameters': f"C={lr_final_params['C']}",
        'Test accuracy': lr_test_metrics['accuracy'],
        'Macro F1': lr_macro_f1,
        'Weighted F1': lr_weighted_f1,
        'Training runtime (s)': lr_fit_time,
        'Number of parameters': lr_final_summary['params']
    },
    {
        'Model': 'MLP',
        'Best hyperparameters': f"alpha={mlp_final_params['alpha']}, hidden_units={mlp_final_params['hidden_units']}",
        'Test accuracy': mlp_test_metrics['accuracy'],
        'Macro F1': mlp_macro_f1,
        'Weighted F1': mlp_weighted_f1,
        'Training runtime (s)': mlp_fit_time,
        'Number of parameters': mlp_final_summary['params']
    },
    {
        'Model': 'CNN',
        'Best hyperparameters': f"kernel_size={cnn_final_params['kernel_size']}, stride={cnn_final_params['stride']}, pool_size={cnn_final_params['pool_size']}, num_filters={cnn_final_params['num_filters']}",
        'Test accuracy': cnn_test_metrics['accuracy'],
        'Macro F1': cnn_macro_f1,
        'Weighted F1': cnn_weighted_f1,
        'Training runtime (s)': cnn_fit_time,
        'Number of parameters': cnn_final_summary['params']
    }
])
```

```

{
    'Model': 'MLP',
    'Best hyperparameters': str(best_mlp_params),
    'Test accuracy': test_accuracy,
    'Macro F1': mlp_macro_f1,
    'Weighted F1': mlp_weighted_f1,
    'Training runtime (s)': final_mlp_runtime,
    'Number of parameters': final_mlp_model.count_params()
},
{
    'Model': 'CNN',
    'Best hyperparameters': str(best_cnn_params),
    'Test accuracy': cnn_test_accuracy,
    'Macro F1': cnn_macro_f1,
    'Weighted F1': cnn_weighted_f1,
    'Training runtime (s)': final_cnn_runtime,
    'Number of parameters': final_cnn_model.count_params()
}
])

display(model_comparison_df)

model_comparison_df.to_csv(output_dir / 'model_comparison_table.csv')
model_comparison_df.to_latex(output_dir / 'model_comparison_table.t

```

	Model	Best hyperparameters	Test accuracy	Macro F1	Weighted F1	Training runtime (s)
0	Logistic Regression	C=0.0001	0.527375	0.467838	0.484070	38.401924
1	MLP	{'n_hidden_layers': 2, 'n_hidden_neurons': 256...}	0.670625	0.664842	0.669377	63.104042
2	CNN	{'conv_filters': 64, 'dense_units': 128, 'drop...	0.835125	0.832509	0.834921	640.515545

5. AI Acknowledgement

Generative AI Tools Used

The primary generative AI tool used throughout this assignment was **ChatGPT** (OpenAI, GPT-5 model).

How Generative AI Tools Were Used

We mainly used AI tools to assist with this assignment by seeking the help for the usage of functions such as Keras and TensorFlow, explaining the concepts behind the models for better understanding, and polish the markdown cells.

